Repository

Season: 4 · Enterprise · Category: Enterprise · Level: Intermediate

Intent

- Collection-like interface over persistence.

Problem

- Without this pattern, code tends toward: New things up with conditionals and duplicated construction.

Real-world Analogy

- Librarian finds books without exposing the stacks.

When to Use

- Microservices / modular monoliths.
- The design force named in Intent shows up more than once.

When Not to Use

- Greenfield CRUD without distribution.
- A simpler language feature (record, sealed switch, function value) already solves it.

Pattern Structure

- See mermaid diagram in explanation.md

Classic Implementation

```java
package com.buildwithwaqar.patterns.enterprise.repository;

import java.util.Collection;
import java.util.Map;
import java.util.Optional;
import java.util.concurrent.ConcurrentHashMap;

record User(String id, String email) {}

interface UserRepository {
    Optional<User> findById(String id);
    User save(User user);
    Collection<User> findAll();
}

/** Collection-like interface over persistence details. */
public final class InMemoryUserRepository implements UserRepository {
    private final Map<String, User> store = new ConcurrentHashMap<>();

    public Optional<User> findById(String id) { return Optional.ofNullable(store.get(id)); }
    public User save(User user) { store.put(user.id(), user); return user; }
    public Collection<User> findAll() { return store.values(); }
}
```

Modern Java 25 / Functional

- Sealed events + pattern matching for projections.

Worked Example

```
See the JUnit test beside the implementation.
Run: mvn -Dtest=*Test test
Focus package matching: enterprise/repository/
```

Pitfalls

- Keep demos minimal — production needs idempotency and observability.
- Saga compensations must be careful.

Interview Q&A

- Q: Production concern for Repository? A: Idempotency, observability, and failure modes.
- Q: How would you test this? A: Prefer injecting interfaces or interpreters; keep demos deterministic (fixed clocks, canned inputs).

Key Takeaways

- Collection-like interface over persistence.
- Prefer Java 25 sealed types / records / virtual threads where they simplify the classic shape.
- Ship working tests with every episode.

Video Script Highlights

- [0:00] Hook — Open with the analogy: Librarian finds books without exposing the stacks. [0:40] Intent — State the design force in one sen...